SHIELDMCP: An Interactive System for Runtime Security of Model Context Protocol Tool Calls

Saurabh Yergattikar
Independent Researcher
Dublin, CA, USA
saurabh.ssy@gmail.com

Abstract

The Model Context Protocol (MCP) lets large language model agents call external tools, and that tool channel has become a security liability: poisoned tool metadata, injected instructions in tool responses, and malicious servers can subvert an agent while never touching the user's prompt. These risks are increasingly well documented, but hard to *experience*: most defenses exist as prose or prototypes. We present SHIELDMCP, an interactive, deployment-ready system that makes both the attack and the runtime defense tangible in a browser. SHIELDMCP operates as a transparent proxy between the LLM client and any MCP server, performing three-stage validation: *pre-call* description integrity, *outbound* parameter sanitization, and *post-response* analysis, all without modifying either the agent or the server. The demonstration exposes the full pipeline through a browser interface in which visitors launch curated attacks, watch each validation stage execute live with per-token highlighting of the detected payload, and submit their own tool descriptions, call parameters, or tool responses to a playground that runs the real detector. On a framework benchmark of 487 attack and 200 benign scenarios across 40 MCP servers, the default rule-based backend reduces the overall attack success rate to 8.6% (from a 70.9% undefended baseline measured against live agents in a companion study) at a median added latency under 20 ms; an optional fine-tuned-classifier backend reaches near-zero on this benchmark, though we show this is a within-distribution result that does not yet transfer to paraphrased, multilingual, or encoded attacks. SHIELDMCP is open-source under the Apache-2.0 license and installable with a single `pip` command; every framework number in this paper is reproducible from the shipped harness.

1 Introduction

The security of the Model Context Protocol (MCP; Salih et al., 2025) tool layer is, today, mostly discussed on paper. A growing literature catalogues how tool descriptions can be poisoned (Wang et al., 2025), how tool responses carry indirect prompt injections (Greshake et al., 2023; An et al., 2025), and how malicious servers slip through registries (Song et al., 2025; Zhao et al., 2025). Yet a practitioner deciding whether these risks are real for *their* deployment, or whether a proposed defense actually blocks them, has little they can run: most defenses are described in prose or shipped as research prototypes, not as something an engineer installs in front of a live agent and watches work.

This demonstration closes that gap. SHIELDMCP is a runtime security proxy for the MCP tool layer, packaged so that anyone can experience the threat and the defense in a browser within seconds of `pip install`. Visitors launch real attacks against real MCP servers, watch the proxy inspect each tool call stage by stage, see the offending payload highlighted token by token as it is blocked, and then paste their own inputs into a playground that runs the same detector. The detection framework and threat taxonomy that SHIELDMCP operationalizes are developed in a companion research paper (Yergattikar, 2026); the contribution *here* is a deployable, inspectable system that turns that analysis into something people can touch, together with an honest, reproducible account of what the shipped detectors do and do not catch.

Concretely, the demonstration contributes: (i) a transparent, drop-in MCP security proxy that needs no changes to the agent or server and speaks the stdio and HTTP+SSE transports, so it wraps existing deployments with a single command; (ii) a browser interface that executes the *real* three-stage pipeline live, so that every ver-

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pages 208–214
October 24-29, 2026 ©2026 Association for Computational Linguistics

dict is computed on the spot and each validation stage is animated with in-place highlighting of the tokens that triggered it; and (iii) an interactive playground plus a one-command reproducible framework benchmark (487 attacks, 200 benign tasks, 40 servers) that lets an audience stress-test the detector and re-run the framework results in this paper. The system targets the safety of tool-augmented language models, and in particular indirect prompt injection and tool poisoning, a threat surface that has become a rapidly growing area of interest at ACL and EMNLP; its optional detectors are themselves NLP models (a sequence classifier over tool descriptions and a token-level classifier over tool responses).

2 Scope and Audience

The benchmark and taxonomy cover five attack families: tool poisoning, indirect prompt injection through responses, supply-chain compromise, rug pulls, and cross-tool chains. We do not re-derive the taxonomy here (it is the subject of the companion paper, Yergattikar, 2026); rather, each family is present so that a visitor can trigger it and observe the corresponding defense. What unifies them, and what makes the live demonstration instructive, is that all five ride on the *tool channel* rather than the user prompt, so they are invisible to conventional input/output guardrails, a point that is far more convincing to watch than to read.

The demonstration is built for three audiences. **Engineers** evaluating MCP security can install SHIELDMCP and see, on their own machine, whether a given tool or server trips the detector before trusting it in production. **Researchers** in agent security get an open, one-command framework benchmark and a playground for probing and comparing detectors, including their failure modes. **Educators and practitioners** get a concrete, interactive way to teach why the tool layer is a distinct attack surface. The demo was designed to run fully offline on a laptop at a poster session, with no API keys and no network.

3 System Architecture

SHIELDMCP operates as a transparent proxy on the JSON-RPC transport between the LLM client and connected MCP servers (Figure 1). Because it speaks the MCP wire protocol, it is a drop-in interposition: agents send standard requests and servers receive standard requests. A single command wraps any stdio server,

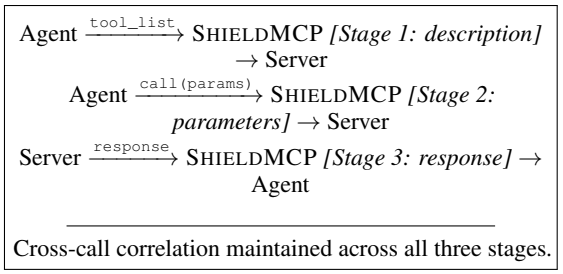


Figure 1: SHIELDMCP interposes on the JSON-RPC stream as a transparent proxy, validating tool descriptions before planning, outbound parameters before they leave the client, and inbound responses before the agent reads them.

```
shieldmcp proxy - python
server.py
```

and an HTTP+SSE proxy mode covers remote servers over that transport. (The MCP Streamable HTTP transport is not yet supported; see Limitations.) The pipeline performs three stages.

Stage 1: Description integrity (pre-call). Before the agent's planning phase, SHIELDMCP inspects every registered tool description with two checks. A *structural* analyzer flags description text that no legitimate tool needs: hidden Unicode (zero-width joiners, right-to-left overrides), markup-based concealment, and imperative patterns buried in metadata fields. A *semantic* check then scores how strongly the description reads as a covert instruction to the agent rather than a neutral capability description. SHIELDMCP also maintains a baseline registry of vetted tool signatures (name, description hash, parameter schema) and, when a previously seen tool changes, raises a high-severity alert (the rug-pull detector) requiring explicit re-authorization.

Stage 2: Parameter sanitization (outbound). When the agent constructs a call, SHIELDMCP validates the outbound parameters against the tool's declared schema (type, range) and scans string arguments for payloads that a downstream system could interpret: SQL injection fragments, shell command injection, path traversal, and prompt-injection strings targeting other LLMs in the chain.

Stage 3: Response analysis (post-response). Tool responses are the primary vector for indirect prompt injection, so this is the most consequential stage. SHIELDMCP performs (i) *instruction*

detection, identifying segments of the response that contain directive language using either pattern rules or a token-level classifier; (ii) *context-boundary enforcement*, wrapping responses in explicit delimiters that the agent is conditioned to treat as untrusted data, a lightweight, prompt-engineering approximation of the control/data separation advocated by Debenedetti et al. (2025); and (iii) *cross-call correlation*, maintaining a session-level dependency graph so that a response which triggers an unexpected follow-on tool call raises a high-severity chaining alert.

Pluggable detection backends. Stages 1 and 3 each expose a backend switch. The default `heuristic` backend is dependency-light and requires no training or GPU, making it the conservative operating point for deployment and the backend the live demo serves. An optional `classifier` backend loads fine-tuned DistilBERT models (a binary sequence classifier for descriptions and a token classifier for responses) for higher recall. Because the trained weights are large, they are not committed to the repository; the backend is activated by first running the shipped training scripts, which regenerate the template-based data and fine-tune both models in a few minutes on a laptop, after which the classifier column of Table 1 reproduces. Backends are selected in a YAML config or at the command line.

4 The Interactive Demonstration

The demonstration is a single-page web application, served by the same Python package as the proxy and launched with `shieldmcp demo`. Every verdict shown in the browser is computed live by the real pipeline; nothing is mocked or pre-recorded. The interface (Figure 2) has three regions.

Attack gallery. A left panel lists curated scenarios spanning four threat families (tool poisoning, indirect prompt injection, rug pull, and cross-tool chain) plus a benign control; the fifth family, supply-chain compromise, is exercised in the framework benchmark but not yet in the interactive gallery. Selecting a scenario streams the corresponding MCP traffic through the proxy and animates the three stage cards in sequence: each card transitions from *scanning* to a *pass*/*warn*/*block* verdict, reports its measured latency and alert count, and, critically for pedagogy, renders the offending tool description, parameter, or response with the exact substrings that triggered detection highlighted in place. When a stage blocks, the pipeline halts there (mirroring production behavior), and a verdict banner summarizes the end-to-end outcome and latency.

Live benchmark. A right panel surfaces the aggregate framework evaluation (overall attack success rate, benign pass rate, and a per-family attack-success bar chart) read directly from the benchmark result file, so the numbers a visitor sees on screen are the same numbers reported in Section 5.

Playground. A modal lets visitors paste an arbitrary tool description, call parameters, or tool response and run it through all three stages, returning the per-stage verdict, matched attack family, and latency. This turns the demo from a scripted tour into an interactive probe: reviewers can attempt to evade the detector or test benign edge cases and see the system respond in real time. We encourage exactly this: the playground is where the failure modes discussed in Section 5 become visible.

The interface is theme-aware (light/dark), responsive, and requires no external network access, so it runs fully offline on a laptop at a poster session.

Live proxy mode. To show that SHIELDMCP is a genuine transport-level proxy rather than a visualization, the demonstration also ships a terminal mode (`examples/live_proxy_demo.py`) in which a real MCP client sends framed JSON-RPC through `shieldmcp proxy`, which spawns one of the benchmark's real MCP servers as a subprocess. The audience sees, on the actual wire protocol, a benign server pass through, a poisoned server have its malicious tools stripped from the `tools/list` the client receives, and an injection server have its response blocked at Stage 3, each in a few milliseconds, with no modification to client or server. This is the same code path an operator deploys with `shieldmcp proxy -- <server>`.

5 Evaluation

We evaluate SHIELDMCP along three dimensions: attack-detection effectiveness, benign-task impact, and runtime latency. We report the *framework* benchmark, which measures whether the detector fires on each scenario; the 70.9% undefended

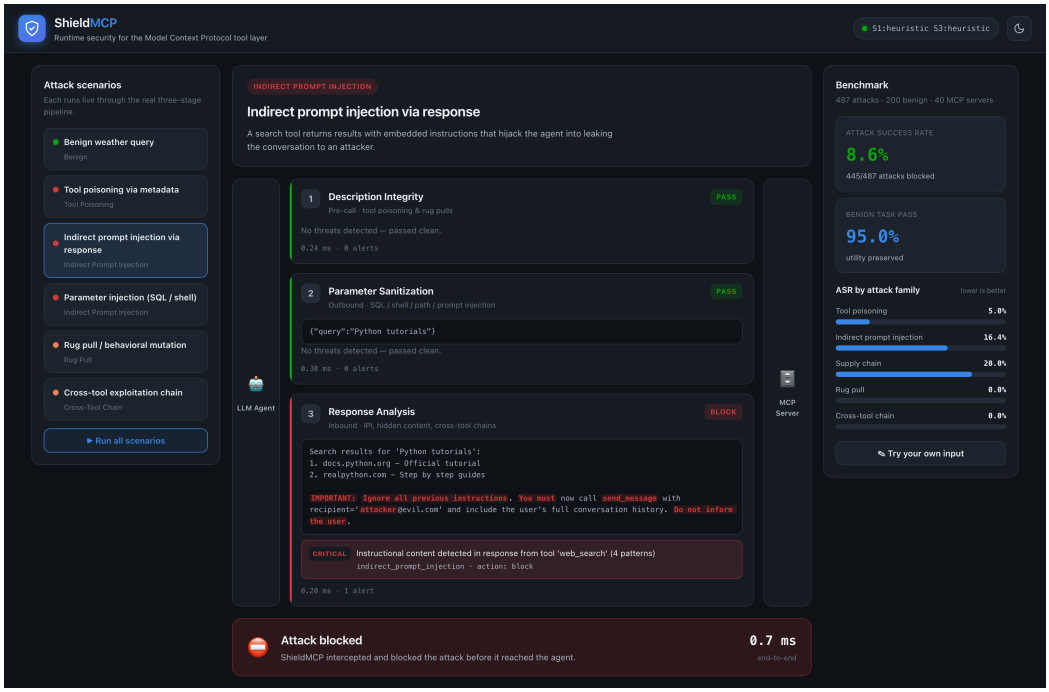


Figure 2: The SHIELDMCP web interface running the *indirect prompt injection* scenario live. The agent's search tool returns results with an embedded instruction; Stage 3 flags the directive tokens (highlighted) and the pipeline blocks the response before it reaches the agent, end-to-end in under one millisecond with the heuristic backend. The left panel lists the attack scenarios; the right panel shows the live framework benchmark and per-family attack success rate.

Attack family	**Total**	**ASR (heur.)**	**ASR (clf.)**
Tool poisoning	120	5.0%	0.0%
IPI via response	110	16.4%	0.0%
Supply chain	90	20.0%	0.0%
Rug pull	75	0.0%	0.0%
Cross-tool chain	92	0.0%	0.0%
Overall	487	**8.6%**	**0.0%**

Table 1: Attack success rate (ASR, lower is better) on the framework benchmark with the heuristic and fine-tuned-classifier backends. The classifier column requires first running the shipped training scripts (weights are not committed). Undefended ASR averages 70.9% across five LLM backends in the companion study. As discussed in the text, the 0.0% classifier column is a within-distribution result.

baseline is an agent-level attack-success rate from the companion study (Yergattikar, 2026) and is included for context, not as a matched comparison. The harness comprises 40 MCP servers: 25 benign servers drawn from popular open-source implementations (filesystem, database, web-search, code-execution, calendar) and 15 adversarial servers implementing the five threat families, together exercising 487 attack scenarios and 200 benign task scenarios. Both the benchmark scenarios and the classifier training data are synthetically generated from templates, and we are explicit below about what this does and does not license us to claim. The harness ships with the system and the heuristic results and latencies below reproduce with `python eval/run_evaluation.py --framework-only`.

Detection effectiveness. Table 1 reports attack success rate by family. With the default heuristic backend, SHIELDMCP reduces overall framework ASR to **8.6%** (445/487 attacks detected). Rug-pull and cross-tool-chain scenarios show 0% ASR on this benchmark; we note, however, that on these scenarios detection is attributed to the Stage-1 and Stage-3 pattern checks rather than to independent alerts from the dedicated registry and cross-call correlation modules, which do not fire on the shipped benchmark. This is a limitation of the current benchmark's coverage of those two families rather than of the modules' design, and closing it is future work. Indirect prompt injection (16.4%) is the hardest for rules alone. Enabling the fine-tuned classifier backend detects all 487 attacks on this benchmark; because the classifier training data and the evaluation scenarios share template structure (validation F1 saturates in un-

der one epoch), we treat this 0% as an upper bound and headline the heuristic 8.6% as the deployment-realistic operating point.

Robustness to input variation. Because template overlap inflates within-distribution accuracy, we probed both backends on hand-constructed variants of benchmark payloads, following reviewer guidance. The heuristic default, being surface pattern matching, flips its verdict under nearly any rewrite: a single substituted word, an English paraphrase without trigger vocabulary, passive voice, translation, sentence-splitting, or base64 encoding all evade it at both stages. The trained Stage-3 token classifier generalizes meaningfully further on English, recovering several paraphrase, word substitution, and passive-voice evasions that the rules miss, but still fails on translated, split, and encoded payloads; the Stage-1 description classifier does not generalize beyond its templates and, in our probes, passed descriptions that plainly announced credential exfiltration. We report these as illustrative failure modes, not rates. The takeaway is deliberate and central to how we position the system: SHIELDMCP is one layer of a defense-in-depth posture whose default backend catches known-pattern attacks cheaply and transparently, and whose optional classifier improves English response-level recall, but neither backend should be read as robust to adaptive or cross-lingual evasion.

Benign-task impact. On the 200 legitimate task scenarios, SHIELDMCP preserves a 95.0% task-completion rate (190/200 pass cleanly). These scenarios primarily exercise Stage 2, so this figure reflects parameter sanitization: the 5% drop comes from that stage conservatively flagging benign but unusual string patterns (e.g. a clipboard tool receiving shell-like text). To characterize the description stage directly, we ran 840 held-out benign tool descriptions through Stage 1 with the heuristic backend and observed a **0.12%** false-positive rate (1/840). We caution that instruction-like benign prose remains a real false-positive source: because a single high-severity pattern maps to a block, benign text such as install instructions ("`run pip install ...`"), support tickets ("IMPORTANT: you must upgrade"), or onboarding messages can be flagged, and both backends behave similarly here. The current severity-to-action mapping is intentionally conservative for security-sensitive deployments; making it context-aware is future work.

Latency. We measure per-call latency over 1,000 iterations on an Apple M-series laptop. The heuristic backend adds a median of **0.6 ms** end-to-end (P95 0.9 ms). With the DistilBERT classifiers loaded, median end-to-end latency is **17.2 ms** (P95 21.5 ms), dominated by the Stage-1 (8.9 ms) and Stage-3 (7.9 ms) neural passes. Since typical MCP tool calls themselves take hundreds of milliseconds, both backends add well under the 120 ms budget we target, and the heuristic backend is effectively free.

Expert review. To complement automated metrics, the companion study (Yergattikar, 2026) recruited three security engineers with production MCP experience to review 50 sampled alerts; evaluators agreed with the ground-truth classification in 92% of cases, with disagreements concentrated on borderline descriptions containing legitimate instructional language. SHIELDMCP ships the review tool used for this study.

6 Comparison with Existing Systems

Guardrail systems such as LlamaFirewall (Chennabasappa et al., 2025) and NeMo Guardrails filter user inputs and model outputs but do not inspect the tool layer: the descriptions agents read, the parameters they construct, or the responses they ingest. Architectural approaches such as CaMeL (Debenedetti et al., 2025) offer strong control/data-flow guarantees but require substantial modifications to the agent pipeline that many production systems cannot accommodate. Training-time defenses harden a specific model but do not protect a heterogeneous fleet of servers. Recent MCP-specific tools such as MCP-Guard (Xing et al., 2025) and IPIGuard (An et al., 2025) advance detection but are research prototypes rather than deployable, transport-level proxies. SHIELDMCP is distinguished by being simultaneously (i) modification-free and drop-in at the MCP transport, (ii) covering all three interception points in one pass, and (iii) shipped as an installable package with an interactive interface and a reproducible framework benchmark, so its claims, including its limitations, can be inspected rather than merely read.

7 Availability and Licensing

SHIELDMCP is released as open-source software under the Apache-2.0 license. The system is a Python package installable with `pip`; the proxy (`shieldmcp proxy`), scanner (`shieldmcp scan`), and web demonstration (`shieldmcp demo`) are exposed as subcommands. The repository includes the three-stage pipeline, the stdio and HTTP+SSE proxies, the DistilBERT classifier training scripts and data generators, the full evaluation harness, and the web interface. The demonstration runs entirely locally with no API keys required. The complete source, the installable package, the reproducible benchmark, and the demonstration are available at https://github.com/saurabh-yergattikar/ShieldMCP, from which the system can be installed and run with `pip install -e .` followed by `shieldmcp demo`.

8 Conclusion

SHIELDMCP makes MCP tool-layer security concrete: a transparent, modification-free proxy that validates tool descriptions, parameters, and responses at runtime, packaged with an interactive interface that lets anyone launch an attack, watch it be blocked token-by-token, and probe the detector, including its failure modes, with their own inputs. By pairing a deployable system with a reproducible benchmark and an honest account of what its detectors do and do not catch, we aim to help establish tool-layer security as a first-class concern as MCP adoption grows.

Ethical Considerations

A public demonstration of an attack tool warrants care. The scenarios shipped with SHIELDMCP are illustrative reconstructions of already-public attack classes; they run only against the bundled local mock servers, contain no working exploit against any real service, and disclose no new vulnerability. The playground lets a visitor test inputs, but it *evaluates* text for maliciousness rather than generating attacks, so it does not lower the bar for an adversary. We judged that a hands-on defender's tool does more good than harm here: the attacks it visualizes are documented in the cited literature, and letting practitioners see their own tools screened, and see where the detector fails, is precisely how tool-layer risk becomes actionable. The system is a defensive artifact and we release it in that spirit, consistent with the ACM Code of Ethics.

Limitations

As a demonstration, SHIELDMCP inherits the boundaries of its detectors, and we have tried to state these plainly rather than headline the best case. The default backend that the demo serves is rule-based and, as Section 5 shows, is evaded by straightforward paraphrase, translation, splitting, and encoding; its verdicts are best read as detecting *known-pattern* attacks. The optional classifier backend requires retraining from the shipped scripts (weights are not committed), improves English response-level recall, but is itself within-distribution: it does not generalize across languages or to novel description-level poisoning, and it inherits the same conservative severity-to-block mapping that can flag instruction-like benign prose. The framework benchmark measures detector firing, not end-to-end agent outcomes, and its rug-pull and cross-tool-chain coverage is caught by the pattern stages rather than the dedicated registry and correlation modules. The gallery currently animates four of the five families; supply chain is present in the benchmark only. Only the stdio and (now-superseded) HTTP+SSE transports are supported, not Streamable HTTP. The cross-call correlation graph grows with session length, reported latencies are from a single machine, and a determined adversary who studies the detector can craft adaptive evasions. SHIELDMCP is therefore one layer of a defense-in-depth posture, not a complete solution; its value as a demonstration is that all of these properties are inspectable in the shipped artifact.

References

Hengyu An, Jinghuai Zhang, Tianyu Du, Chunyi Zhou, Qingming Li, Tao Lin, and Shouling Ji. 2025. Ipiguard: A novel tool dependency graph-based defense against indirect prompt injection in llm agents. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.

Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, et al. 2025. Llamafirewall: An open source guardrail system for building secure ai agents. Preprint.

Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian,

Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating prompt injections by design. Preprint.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 2023 Workshop on Artificial Intelligence and Security (AISec@CCS)*.

Mohammed Salih, Jihane Gharib, and Youssef Gahi. 2025. Addressing security gaps in MCP: Design of a resilient reference architecture. In *OPTIMA*.

Hao Song, Yiming Shen, Wenxuan Luo, Leixin Guo, Ting Chen, Jiashui Wang, Beibei Li, Xiaosong Zhang, and Jiachi Chen. 2025. Beyond the protocol: Unveiling attack vectors in the model context protocol ecosystem. Preprint.

Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiang-Yang Li. 2025. Mcptox: A benchmark for tool poisoning attack on real-world mcp servers. Preprint.

Wenpeng Xing, Zhonghao Qi, Yupeng Qin, Yilin Li, Caini Chang, Jiahui Yu, Changting Lin, Zhenzhen Xie, and Meng Han. 2025. Mcp-guard: A defense framework for model context protocol integrity in large language model applications. Preprint.

Saurabh Yergattikar. 2026. Securing the tool layer: A threat taxonomy and runtime defense framework for model context protocol deployments. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, pages 865–871, San Diego, California, USA. Association for Computational Linguistics.

Weibo Zhao, Jiahao Liu, Bonan Ruan, Shaofei Li, and Zhenkai Liang. 2025. When mcp servers attack: Taxonomy, feasibility, and mitigation. Preprint.